

Physics-Grade Benchmark Packaging — DQI Pricing Benchmark Results & Validation

Pricing-to-DQI mini pipeline, paper-faithful DQI microscope, and industrial ILP positioning

This document presents the numerical results that complete the DQI Pricing Benchmark project. Each section corresponds to one deliverable: (1) exact pricing-model construction and pricing-to-artifact handoff, (2) a paper-faithful DQI microscope aligned with the five-step algorithm of Decoded Quantum Interferometry (DQI), (3) a reviewer-facing exact-reference versus WHT encoding benchmark, (4) decoder, postselection, and structure-regime analysis, (5) resource estimation and correctly scoped Qiskit structural-subset validation, and (6) a dynamic-pricing extension plus scaling boundary.

The document deliberately follows the more polished “results-and-validation” packaging style of the continuum-physics reference note: each major result is introduced by a setup section, followed by quantitative results, interpretation, and an explicit verdict. The scientific framing is additionally aligned to two DQI papers. Relative to **Jordan et al., arXiv:2408.08292**, this benchmark adopts the five-step DQI structure, distinguishes paper-derived versus heuristic weight choices, and treats decoder/postselection behavior as a first-class metric. Relative to **Sabater et al., arXiv:2509.08328**, it adopts the industrially relevant pricing framing and the ILP → max-XORSAT → DQI narrative, but keeps the implementation intentionally small-scale, exact-verification-heavy, and reviewer-facing rather than claiming full industrial deployment.

Validated against 14 uploaded notebooks with persisted outputs (review date: 2026-03-17)

01_reproduce_toy_dqi.ipynb, 02_pricing_instance_demo.ipynb, 03_paper_faithful_dqi.ipynb, 04_pricing_pipeline.ipynb, 05_decoder_comparison.ipynb, 06_resource_analysis.ipynb, 07_qiskit_structural_parity.ipynb, 08_paired_encoding_comparison.ipynb, 09_3period_pricing_extension.ipynb, 09_decoder_phase_diagram.ipynb, 10_alpha_finite_size_validation.ipynb, 11_decision_faithfulness_dashboard.ipynb, 12_quality_vs_cost.ipynb, and 13_scaling_analysis.ipynb.

Methodological note. This report is based on the persisted notebook outputs uploaded in this conversation. It does **not** claim a fresh rerun beyond those saved outputs. All 14 notebooks contain saved outputs and no stored code-cell error outputs.

Repository map and benchmark taxonomy

The benchmark is organized as a layered research artifact: interactive notebooks at the top, orchestrators and runners beneath them, then the DQI core, classical baselines, decoders, encoding backends, and the problem/instance generators that define the benchmark families. The architecture below is restored here because it gives reviewers a compact map from claimed results to implementation loci.

NOTEBOOKS 01-13: Interactive exploration, demos, analysis	
ORCHESTRATION benchmark.py · matrix_runner.py · encoding_compare.py	
PIPELINE dqi_pipeline.py · resources.py · metrics_decision.py	
DQI CORE └─ dqi_state.py └─ dqi_circuit_qiskit.py └─ dicke_circuit.py	CLASSICAL BASELINES └─ classical_baselines.py └─ pricing_model_ortools.py └─ *_multiperiod.py variants
DECODERS decoder_bruteforce.py · decoder_bp1.py · decoder_bp_osd.py	
ENCODING reduction.py (WHT) · ilp_to_xorsat_exact.py (ILP-derived exact)	
WEIGHTS / ALPHA weights.py · weights_paper.py · weights_heuristic.py	
PROBLEM DOMAIN problem_generator.py · pricing_ilp.py · family_s_generator.py	
DATA & ARTIFACTS data/instances/ · results/runs/ · figures/	

Benchmark classes and families

This benchmark uses **families** to separate semantically different validation regimes and **classes** to separate instance scales or structural regimes inside those families.

Label	Kind	Definition	Primary role in the report
P3–P7	Pricing class	Frozen application-style pricing instances with 3–7 features, encoded as 6–14 binary variables via paired-bit tier encoding	Main benchmark ladder for exact baselines, scaling, resources, and pricing-quality claims
C1–C3	Coherent validation class	Tiny fully specified DQI-compatible artifacts where exact statevector and distribution-level checks remain tractable	Faithful validation of the five-step DQI construction, alpha modes, and zero-regret coherent checks
S_*	Structure family	Synthetic parity-system instances that vary matrix density, rank, and collision structure independently of pricing semantics	Decoder-phase studies, structure/postselection analysis, and controlled stress tests
wht_truncated	Encoding class	Walsh–Hadamard top-k surrogate path from truth table to sparse parity artifact	Exploratory reduction path; useful for diagnostics and ablations
ilp_derived	Encoding class	Exact ILP-derived pricing-to-XORSAT path with no surrogate approximation	Mainline reviewer-facing encoding because it preserves objective ordering and removes surrogate error
Matrix A	Reporting matrix	Paired encoding benchmark over the pricing ladder (P3–P7)	Compares exact-reference versus WHT-style encoding quality
Matrix B	Reporting matrix	Exact-reference / alpha-validation benchmark over the coherent family (C1–C3)	Supports tiny exact checks and alpha/coherent validation
Matrix C	Reporting matrix	Structure-sensitive decoder benchmark over S_* plus selected pricing-translated rows	Summarizes decoder and postselection behavior by parity structure
low_rank_full, mid_rank_deficient, etc.	Structure-regime class	Bucketized descriptors over parity artifacts based on density/weight regime and matrix rank status	Compact summary language for decoder-phase results

A few definitions are worth stating explicitly.

- **Reporting matrices (A, B, C).** In this report, “Matrix A/B/C” refers to **benchmark result matrices**—that is, grouped evaluation panels—not to a single linear-algebra object used inside the DQI circuit. They organize the validation evidence by question being asked.
- **Matrix A.** The **paired encoding matrix** over the pricing ladder. It compares the `wht_truncated` surrogate path against the `ilp_derived` / exact-reference path on pricing instances (P3–P7). The point of Matrix A is to answer: *does the reduction preserve pricing-objective ordering and decision quality?*
- **Matrix B.** The **exact-reference / alpha-validation matrix** over the tiny coherent family (C1–C3). It collects small-instance checks where amplitudes, support, regret, alpha schedules, and coherent-versus-mixture behavior can be verified exactly. The point of Matrix B is to answer: *is the paper-faithful DQI construction behaving correctly in the regime where exact inspection is still possible?*
- **Matrix C.** The **structure-sensitive decoder matrix** over synthetic structure instances (S_*) together with selected translated pricing rows. It groups results by sparsity/density, rank, collisions, and decoder choice (brute force, BP1, BP-OSD, oracle where applicable). The point of Matrix C is to answer: *which parity-system structures are favorable or unfavorable for decoding and postselection?*
- **Pricing classes (P3–P7).** These are the frozen benchmark instances of the pricing problem family. P3 means 3 pricing features (6 binary variables after paired-bit encoding), P4 means 4 features (8 bits), and so on through P7 (14 bits). They are the most interpretable instances because every score still maps back to a concrete pricing decision.
- **Coherent classes (C1–C3).** These are tiny exact-validation artifacts used when the goal is not application realism but strict correctness. They are small enough that one can inspect amplitudes, support, regret, and postselection behavior without approximation ambiguity.
- **Structure family (S_*).** These instances intentionally suppress pricing semantics and instead vary the algebraic properties of the parity system (B, v): sparsity/density, row and column weights, rank deficiency, and collision structure. They answer the question “what kinds of parity structure are easy or hard for a given decoder?” rather than “what pricing decision is best?”
- **Encoding classes.** The benchmark distinguishes the **WHT surrogate path**, which approximates the pricing objective through a truncated Walsh–Hadamard expansion, from the **ILP-derived exact path**, which preserves the target objective exactly in the benchmarked reference conversion.
- **Structure-regime classes.** Labels such as `low_rank_full` or `mid_rank_deficient` are analysis buckets, not separate datasets. They summarize whether an artifact lies in a low/mid/high structural-density regime and whether its parity matrix is full rank or rank deficient.

Why these families are separated

The report makes stronger claims precisely because the families are not mixed together indiscriminately.

Family / class	What it validates	What it does not by itself validate
P3-P7	End-to-end pricing relevance, exact baselines, scaling/resource trends	Pure decoder-mechanism claims independent of problem semantics
C1-C3	Faithful small-instance correctness of the DQI construction	Application-scale usefulness
S_*	Decoder sensitivity and parity-structure effects	Direct pricing-quality conclusions
wht_truncated vs ilp_derived	Reduction quality and objective faithfulness	Decoder-independent physical correctness
Structure-regime buckets	Aggregate decoder trends	Instance-specific causal proof

Notebook quick reference

#	Name	Purpose	Key Outputs
01	reproduce_toy_dqi	Canonical DQI reproduction	Histogram figure
02	pricing_instance_demo	Human-readable verification	Instance tables
03	paper_faithful_dqi	Appendix: exact microscope	Small-instance DQI diagnostics
04	pricing_pipeline	CP-SAT $\rightarrow$ surrogate bridge	Artifact export
05	decoder_comparison	BF vs BP1 sensitivity	Decoder metrics
06	resource_analysis	Resource/cost diagnostics	Cost tables
07	qiskit_structural_parity	Qiskit subset checks	Parity report
08a	paired_encoding_comparison	ILP-exact vs WHT	Comparison table
08b	coherent_validation	Coherent mode validation	Validation report
09a	decoder_phase_diagram	Decoder phase analysis	Phase diagram
09b	3period_pricing_extension	Multi-period extension	Extended results
10	alpha_finite_size_validation	Appendix: alpha validation	Finite-size alpha appendix
11	decision_faithfulness_dashboard	Faithfulness metrics	Dashboard
12	quality_vs_cost	Tradeoff analysis	Pareto plots
13	scaling_analysis	Scaling behavior	Scaling curves

Source modules (src/)

Core DQI implementation

File	Description
dqi_state.py	Five-step DQI statevector preparation implementing Jordan et al. Steps 1–5: Dicke state, phase oracle, Hadamard/interference, syndrome extraction, and projection logic
dqi_pipeline.py	Full sampling workflow: state preparation $\rightarrow$ decode $\rightarrow$ evaluate $F(x)$ and $G(x)$ $\rightarrow$ collect metrics
dqi_circuit_qiskit.py	Structural Qiskit port for hardware-pathway validation; statevector-equivalent subset, not a claim of full optimized deployment
dicke_circuit.py	Analytical resource formulas and/or construction rules for Dicke state preparation on the validated small-instance regime

Problem domain

File	Description
problem_generator.py	Discrete package-pricing generator: features, tiers, penalties, and demand constraints over a finite configuration space
problem_generator_multiperiod.py	3-period extension with shared-segment demand shifts and time-coupled price choices
pricing_ilp.py	Canonical ILP model contract for the toy pricing family
pricing_model_ortools.py	CP-SAT interface used as the exact classical baseline and source of solve semantics
pricing_model_ortools_multiperiod.py	CP-SAT backend for the 3-period extension

Encoding / reduction

File	Description
reduction.py	WHT path: Walsh–Hadamard transform $\rightarrow$ top-k Fourier truncation $\rightarrow$ max-XORSAT surrogate
ilp_to_xorsat_exact.py	Exact path: ILP-derived reference encoding with no surrogate approximation
encoding_compare.py	Runner for paired WHT vs ILP-exact comparisons
verify_encoding.py	Validation helpers for encoding correctness and artifact consistency

Decoders

File	Method	Use case
decoder_bruteforce.py	Bounded-distance enumeration	Exact reference for very small m , n
decoder_bp1.py	Hard-decision bit-flip	Fast heuristic and main practical comparator
decoder_bp_osd.py	BP + ordered-statistics decoding	Improved heuristic on harder parity artifacts
decoder_oracle.py	Exact oracle	Tiny instances only
decoder_study_runner.py	Batch runner	Structure-regime and decoder-sensitivity studies

Weights / alpha

File	Description
weights.py	Compatibility shim exporting <code>uniform_weights</code> , <code>heuristic_alpha</code> , and <code>paper_alpha</code>
weights_paper.py	Paper-derived finite tridiagonal matrix construction for the <code>\alpha_t</code> profile
weights_heuristic.py	Proxy-matrix heuristic (kept explicitly distinct from paper-faithful weights)
alpha_theory_validation.py	Finite-size validation runner on the coherent family

Classical baselines

File	Methods
classical_baselines.py	Brute force, CP-SAT (via OR-Tools), simulated annealing
classical_baselines_multiplier.py	Same baseline family for the 3-period extension

Analysis & reporting

File	Description
benchmark.py	Main orchestrator: runs DQI plus classical baselines on frozen instances
matrix_runner.py	CLI for P, C, and S family matrix runs with standardized outputs
resources.py	DQI resource estimation: gate counts, qubit requirements, and depth estimates
resources_compare.py	Quality-vs-cost join layer that assembles existing benchmark outputs
structure_metrics.py	Parity-system diagnostics: row/column weights, rank, collision proxies
metrics_decision.py	Shared decision-quality metrics such as regret, top-k success, and faithfulness summaries
faithfulness.py	Decision-faithfulness normalization and aggregation logic
report_schema.py	Schema validators for the reporting layer
boundary.py	Execution-boundary record capture for executed vs analytic-only claims
notebook_utils.py	Shared notebook execution and display helpers

Instance families

File	Description
coherent_reference.py	Loader for coherent tiny-instance artifacts (C1–C3)
family_s_generator.py	Synthetic structure-sweep generator over density, collision, and rank variations
family_s_manifest.py	Validator for Family S artifacts and manifests

Data layout (data/)

```

data/
├── instances/
│   ├── pricing_3feat_6bit.json    # Frozen pricing: 3 features → 6 bits
│   ├── pricing_4feat_8bit.json    # Frozen pricing: 4 features → 8 bits
│   ├── pricing_5feat_10bit.json   # Frozen pricing: 5 features → 10 bits
│   ├── pricing_6feat_12bit.json   # Frozen pricing: 6 features → 12 bits
│   ├── pricing_7feat_14bit.json   # Frozen pricing: 7 features → 14 bits
│   ├── surrogate_*.json          # WHT-truncated surrogates
│   ├── coherent/
│   │   ├── C1.json, C1.npz        # Coherent tiny instances
│   │   ├── C2.json, C2.npz
│   │   ├── C3.json, C3.npz
│   │   └── coherent_manifest.json
│   └── family_s/
│       ├── S_dense_low_collision.json
│       └── S_sparse_high_collision.json

```

```

|   |   |   S_balanced_rank_full.json
|   |   |   ... (8 variants)
|   |   |   manifest.json

```

What these artifacts represent

- **pricing_*feat_*bit.json** are the main frozen application-style benchmark instances. The feature count is the business-facing size; the bit count is the encoded optimization size.
- **surrogate_*.json** are exploratory WHT-derived artifacts used when the benchmark evaluates reduction quality rather than exact encoding fidelity.
- **coherent/** stores the exact tiny-instance validation family that supports clean amplitude- and support-level checks.
- **family_s/** stores the structure-only stress-test family where decoder behavior can be studied independently of pricing semantics.

Output layout (results/, figures/)

```

results/
├── runs/
│   └── YYYYMMDD_HHMMSS/           # Timestamped run outputs
│       ├── benchmark_*.json       # Per-instance results
│       ├── resources_scaling.json
│       └── run_manifest.json
├── run_logs/                      # Execution logs
└── reports/
    └── artifact_index.md

```

```

figures/
└── notebook01_toy_histogram.png    # Generated visualizations

```

These output layers separate **raw run evidence** from **reporting artifacts**. That distinction matters because the report intentionally refers to persisted notebook outputs and run products rather than implying unrestricted reruns.

Tests (tests/)

Pattern	Count	Coverage
test_dqi_*.py	4	DQI state, pipeline, coherent modes
test_decoder*.py	5	All decoder variants
test_encoding*.py	2	Encoding paths
test_weights*.py	2	Alpha computation
test_pricing*.py	3	Pricing model, pairing
test_notebook*.py	12	Notebook smoke tests
test_stage_*.py	6	Stage-specific validation
test_resources*.py	3	Resource estimation
test_*_multiperiod.py	3	Multi-period variants
Other	~26	Baselines, metrics, schema, notebook utilities, and glue code

Total: approximately **67 test files**.

The testing layout mirrors the benchmark-family split: correctness tests on the DQI core, benchmark-family tests on pricing/coherent/structure slices, and reporting/schema tests that protect the packaging layer.

Configuration

File	Purpose
pyproject.toml	Package metadata, dependencies, and pytest configuration
README.md	Project overview, quick start, usage disclaimers, and benchmark scope
Theoretical_Framework.md	Mathematical foundations and DQI framing notes

Executive summary

Question	Notebook evidence	Quantitative result	Verdict
Is the pricing baseline exact and well-defined?	02, 04, 13	CP-SAT matches brute force on P3, P4, P5; P5 optimum $F^* = 3000.0$, solve time 0.0489 s in Notebook 04	
Which encoding should be the reviewer-facing default?	08, 13	On 18 matched pricing pairs, <code>ilp_derived</code> beats <code>wht_truncated</code> : mean <code>best_F</code> = 2933.33 vs 2844.44; mean <code>top1_regret</code> = 0.0 vs 88.89; mean $\rho(F, G) = 1.0000$ vs 0.9646	
Does decoder choice matter?	05, 09, 13	On the 10-bit pricing artifact at <code>ell = 3</code> , BP1 reaches <code>best_F</code> = 2900.0 with success 0.769597 , versus brute force 2300.0 and 0.075275 ; on the exact-statevector P4 slice, BP1 reaches 2800.0 vs brute force 2700.0	
Is there honest Qiskit evidence?	07	Tiny structural-subset parity check gives $TVD = 0.000e+00$ and max absolute probability difference $0.000e+00$	
Are resources and scope boundaries clearly stated?	06, 13	P3→P7 analytic resource estimate rises from 48 to 65 qubits and depth from 2953 to 12848; P7 is analytic-only	
Does the dynamic-pricing extension show nontrivial behavior?	09_3period	Dynamic horizon uplift ranges from 0 to 150 across the 9 frozen scenarios	
Is the alpha appendix now usable?	10	48 candidate rows examined, 42 retained after completeness gate, 6 excluded; C1/C2/C3 all show zero-regret verdicts across retained alpha modes	~
Is there a scaling story beyond P5?	13	Pricing family is extended to P6 ($n=12$, $F^*=3360$) and P7 ($n=14$, $F^*=3100$), with P7 kept analytic-only	

1. Pricing-to-DQI benchmark: exact pricing model and artifact handoff

Code: `problem_generator.py`, `pricing_model_ortools.py`, `classical_baselines.py`, `04_pricing_pipeline.ipynb`

1.1 Setup and benchmark family

The **P family** is the automotive feature-pricing benchmark family used throughout the notebook suite. Each instance defines **N** features with three tiers encoded in 2 bits per feature, customer-segment demand rules, bundle constraints, and a luxury-cap rule. The result is a compact but realistic combinatorial pricing model: small enough for exact verification, yet structured enough to support ILP-style modeling and DQI-compatible reductions.

Instance	Features	Bits n	Total configs	Feasible	Feasible %	Optimal F^*	Validation scope
P3	3	6	64	23	35.94%	3000.0	analytic + executed
P4	4	8	256	57	22.27%	2800.0	analytic + executed
P5	5	10	1024	154	15.04%	3000.0	analytic + executed
P6	6	12	4096	520	12.70%	3360.0	analytic + executed
P7	7	14	16384	1272	7.76%	3100.0	analytic-only

Feature growth across the family is deliberately incremental:

Instance	Feature set
P3	Heated Seats, Panoramic Roof, Premium Audio
P4	P3 + Adaptive Cruise
P5	P4 + Ambient Lighting
P6	P5 + Head-Up Display
P7	P6 + one additional seventh feature

This family mirrors the *industrial-problem first* philosophy of the automotive DQI paper, but on a much smaller, exactly checkable benchmark scale.

1.2 Exact pricing baseline

Notebook 02 establishes the static pricing benchmark and checks the exact classical baseline.

Instance	Bits	Feasible states	Valid fraction	Optimal F*	CP-SAT agrees with brute force?
3-feature pricing	6	23 / 64	0.359375	3000.0	Yes
4-feature pricing	8	57 / 256	0.222656	2800.0	Yes
5-feature pricing	10	154 / 1024	0.150391	3000.0	Yes

Constraint-pressure diagnostics from the same notebook make the feasibility structure explicit:

Instance	Invalid-tier count	Bundle violations	Luxury-cap violations
3-feature	37	8	1
4-feature	175	60	13
5-feature	781	240	106

1.3 Pricing-to-artifact handoff

Notebook 04 is the key “industrial-style handoff” notebook. It freezes the exact solver output, exports the paired artifacts, and records the encoding interface cleanly.

Field	Saved value
Loaded instance	pricing_5feat_10bit.json
Problem size	n_features = 5, n_bits = 10, n_configs = 1024
CP-SAT status	optimal
Optimal bitstring	0000010000
True pricing objective	F* = 3000.0
Encoded solver objective	G* = 191.0
Brute-force match	True
Solve time	0.0489 s
Exported encodings	['ilp_derived', 'wht_truncated']

Selected P5 optimal package:

Feature	Tier	Price
Heated Seats	Standard	500.0
Panoramic Roof	Standard	800.0
Premium Audio	Premium	800.0
Adaptive Cruise	Standard	600.0
Ambient Lighting	Standard	300.0

1.4 Interpretation

This stage is the benchmark’s most application-relevant component. It reproduces, in miniature, the modeling arc emphasized in the industrial DQI paper: formulate a pricing problem, solve the exact classical reference, and hand off a DQI-compatible artifact. The crucial difference is scope. Instead of claiming a full general ILP-to-LDPC implementation, this repo freezes a compact, inspectable, exact-reference pipeline suitable for notebook-level validation and reviewer scrutiny.

Verdict: The pricing baseline is exact, transparent, and already packaged in the most useful reviewer-facing form: CP-SAT and brute force agree on P3–P5, and the pricing-to-artifact export is explicit and reproducible.

2. Paper-faithful DQI microscope: alignment to Jordan et al.

Code: dq_i_state.py, dq_i_pipeline.py, weights_paper.py, weights_heuristic.py, decoder_bruteforce.py, decoder_bp1.py, 01_reproduce_toy_dqi.ipynb, 03_paper_faithful_dqi.ipynb

2.1 DQI structure adopted in this benchmark

The benchmark follows the five-step DQI structure introduced for max-XORSAT: prepare a superposition over Dicke states, apply the problem-specific phase, compute the syndrome $B^T y$, decode/uncompute the error register, and apply the final Hadamards before measurement. The notebook suite uses this structure as the “paper-faithful” microscope layer rather than treating DQI as a black-box sampler.

The implementation also mirrors two theoretical distinctions from Jordan et al.:

1. **Weight choice matters.** In the regime $2\epsilon_{ll} + 1 < d_{\text{perp}}$, the paper shows that the choice of polynomial weights can be obtained from the principal eigenvector of the matrix $A(m, \epsilon_{ll}, d)$. Once that minimum-distance condition is no longer guaranteed, the

same eigenvector remains a reasonable but no longer provably optimal choice.

2. **Imperfect decoding matters.** When decoding is imperfect, the output state after uncomputation contains both correctly and incorrectly decoded sectors; successful postselection keeps only the correctly uncomputed component. This makes decode success and postselection probability physically meaningful metrics, not bookkeeping extras.

Those two distinctions motivate the repo’s split between **paper**, **heuristic**, and **uniform** alpha modes, and between brute-force and BP-style decoders.

2.2 Notebook 01 — toy DQI reproduction

Notebook 01 is a compact 4-bit / 6-term reproduction of the five-step DQI procedure.

Quantity	Value
n_bits	4
m	6
Postselection success probability	0.5000
Unique sampled states	16 / 16
Best toy bitstring in exact table	1011
Satisfied clauses for best toy bitstring	5

The toy sample distribution includes the true optimum **1011** with probability **0.079167**, but it is not the most probable sampled state. That is acceptable at this stage: the notebook is a pedagogical reproduction layer rather than the mainline benchmark.

2.3 Notebook 03 — exploratory microscope on pricing instances

Notebook 03 runs the paper-faithful DQI pipeline on progressively larger pricing artifacts. It is scientifically useful but should remain explicitly labeled as an **exploratory microscope** rather than the mainline pricing result.

Instance	n	m	ell	F*	Best sampled F (uniform)	Best sampled F (eigenvector)	Approx. ratio	Uniform success	Eigen success	rho(F,G)	Verdict
6-bit	6	12	1	3000.0	3000.0	3000.0	1.0	1.000000	1.000000	0.874636	best_F
8-bit	8	12	2	2800.0	2800.0	2800.0	1.0	0.530303	0.440832	0.895161	best_F
10-bit	10	15	3	3000.0	2400.0	2400.0	0.8	0.075275	0.072415	0.900763	best_F

Alpha-mode comparison on the tiny-supported slice:

Run key	Alpha mode	Execution mode	Best F	Approx. ratio	Success probability	Status
dqi_uniform_mixture	uniform	mixture	3000.0	1.0	0.678571	completed
dqi_paper_mixture	paper	mixture	3000.0	1.0	0.571429	completed
dqi_heuristic_mixture	heuristic	mixture	3000.0	1.0	0.613977	completed
dqi_paper_coherent	paper	coherent	—	—	—	unsupported (coherent_hilbert_cap_exceeded)

2.4 Interpretation

The microscope layer does exactly what a trustworthy DQI validation layer should do: it makes the success boundary visible. The 6-bit instance is cleanly solved, the 8-bit instance remains competitive but loses postselection mass, and the 10-bit instance degrades to **best_F = 2400.0** with success below **0.08**. The point is not that this notebook “wins” at P5, but that it faithfully exposes where decode difficulty and statevector cost begin to dominate.

Verdict: The benchmark contains a genuinely paper-faithful DQI microscope, and it is honest about where that microscope stops being strong on the pricing artifacts.

3. Mainline encoding benchmark: exact-reference vs WHT

Code: `ilp_to_xorsat_exact.py`, `reduction.py`, `encoding_compare.py`, `08_paired_encoding_comparison.ipynb`, `13_scaling_analysis.ipynb`

3.1 Setup

This is the benchmark’s clearest reviewer-facing comparison. Two reduction paths are compared side by side:

- **ilp_derived / exact-reference:** the mainline backend, intended to preserve pricing decisions and objective structure exactly on the validated slice.
- **wht_truncated:** an exploratory Walsh–Hadamard truncation surrogate, retained because it is diagnostically informative but not suitable as the default presentation layer.

This division also clarifies the relationship to the two papers. The exact-reference path plays the role of a faithful problem-to-artifact transfer in the industrial ILP framing, while the WHT path is best interpreted as an exploratory surrogate consistent with the “microscope” philosophy.

3.2 Matched pricing comparison (Notebook 08)

Notebook 08 compares the two encodings on **18 matched pricing records**.

Metric	wht_truncated	ilp_derived	Mean delta (ilp - wht)
Mean best_sampled_F	2844.444444	2933.333333	+88.888889
Mean top1_regret	88.888889	0.0	-88.888889
Mean topk_regret	458.333333	0.0	-458.333333
Mean best_top_G_sampled_F	2475.0	2933.333333	+458.333333
Mean rho(F,G)	0.964604	1.0	+0.035396
Mean retained energy eta	0.975758	1.0	+0.024242
Mean decision distortion	0.029630	0.0	-0.029630

Instance-level summary:

Instance	Backend	Completed rows	Best sampled F	Top-1 regret	Top-k regret	rho(F,G)	Retained energy	Decision distortion
P3	ilp_derived	6	3000.0	0.0	0.0	1.000000	1.000000	0.000000
P3	wht_truncated	6	3000.0	0.0	0.0	0.938027	0.994357	0.000000
P4	ilp_derived	6	2800.0	0.0	0.0	1.000000	1.000000	0.000000
P4	wht_truncated	6	2800.0	0.0	733.333333	0.973945	0.968077	0.000000
P5	ilp_derived	6	3000.0	0.0	0.0	1.000000	1.000000	0.000000
P5	wht_truncated	6	2733.333333	266.666667	641.666667	0.981838	0.964842	0.088889

3.3 Encoding quality versus scale (Notebook 13)

Notebook 13 extends the same comparison through P6 and P7.

Instance	Encoding	rho(F,G)	Retained energy eta	Decision distortion	Top-1 regret	Top-k regret
P3	ilp_derived	1.000000	1.000000	0.000000	0.0	0.0
P3	wht_truncated	0.940113	0.995235	0.000000	0.0	0.0
P4	ilp_derived	1.000000	1.000000	0.000000	0.0	100.0
P4	wht_truncated	0.756300	0.977745	0.000000	0.0	100.0
P5	ilp_derived	1.000000	1.000000	0.000000	0.0	0.0
P5	wht_truncated	0.082844	0.908265	0.000000	0.0	0.0
P6	ilp_derived	1.000000	1.000000	0.077381	260.0	260.0
P6	wht_truncated	-0.015719	0.883544	0.077381	260.0	260.0
P7	ilp_derived	1.000000	1.000000	0.010000	31.0	93.0
P7	wht_truncated	-0.134089	0.818708	0.010000	31.0	93.0

3.4 Interpretation

This is the strongest single argument in the report. The exact-reference path is not merely slightly better; it is categorically more faithful. It keeps $\text{rho}(F,G) = 1.0$ and $\text{eta} = 1.0$ across the entire validated family, while WHT progressively loses objective fidelity and eventually anti-correlates with the pricing objective at P7. That makes the presentation choice straightforward: WHT should be retained as a diagnostic or exploratory reduction, not the benchmark’s default backend.

Verdict: The reviewer-facing default should be the exact-reference / ILP-derived encoding, with WHT clearly labeled exploratory.

4. Decoder sensitivity, postselection, and structure dependence

Code: decoder_bruteforce.py, decoder_bp1.py, 09_decoder_phase_diagram.ipynb, 05_decoder_comparison.ipynb, 13_scaling_analysis.ipynb

4.1 Why decoder metrics matter here

One of the most important lessons from Jordan et al. is that decoder imperfections do not merely reduce accuracy in an abstract way; they directly reshape the postselected DQI state. In that framework, the correctly decoded and misdecoded error sectors contribute differently to the renormalized output after postselection. That is why this benchmark reports **best sampled F**, **postselection success probability**, and **decoder regime sensitivity** together.

4.2 Pricing-artifact decoder comparison (Notebook 05)

Notebook 05 is the clearest single-instance decoder result on the pricing benchmark.

Decoder	e1l	Success probability	Best F	Mean F	Reading
bruteforce	1	1.000000	3000.0	-90994.68	matches optimum
bp1	1	1.000000	3000.0	-90994.68	matches optimum
bruteforce	2	0.571429	3000.0	-107839.31	still reaches optimum
bp1	2	0.911111	3000.0	-105815.33	much higher success
bruteforce	3	0.075275	2300.0	-160322.38	sharp degradation
bp1	3	0.769597	2900.0	-112252.98	substantially more robust

Alpha-mode stability within the same notebook:

Decoder	Best F across heuristic / paper / uniform	Success probability range across alpha modes
bp1	always 3000.0	0.903881 to 0.933333
bruteforce	always 3000.0	0.428571 to 0.571429

4.3 Structure-regime phase diagram (Notebook 09)

Notebook 09_decoder_phase_diagram.ipynb adds matrix-level decoder evidence.

Coverage summary:

Field	Value
Rows in focus	51
Completed	47
Not applicable	4
Family P rows	16
Family S rows	35
Decoders represented	bp1, bruteforce, oracle
Execution filter	mixture_only

Selected structure-regime means:

Decoder	Structure regime	n	Mean best sampled F	Mean postselection success	Mean gain over brute force
bp1	'high	rank_deficient'	9	6.666667	0.533642
bruteforce	'high	rank_deficient'	6	6.000000	0.085340
bp1	'low	rank_full'	6	1005.333333	0.836806
bruteforce	'low	rank_full'	6	1005.333333	0.951852
bp1	'mid	rank_deficient'	4	1553.000000	0.850768
bruteforce	'mid	rank_deficient'	4	1529.750000	0.289305
bp1	'mid	rank_full'	4	2637.500000	0.913045
bruteforce	'mid	rank_full'	4	2450.000000	0.320783

4.4 Scaling corroboration (Notebook 13)

Notebook 13's exact-statevector slice supports the same direction:

Instance	n	e1l	Decoder	Postselection success	Best F	Top-1 regret	Wall-clock (s)
P3	6	1	bp1	1.000000	3000.0	0.0	0.629699
P3	6	1	bruteforce	1.000000	3000.0	0.0	0.826408
P4	8	2	bp1	0.954545	2800.0	0.0	2.885672
P4	8	2	bruteforce	0.454545	2700.0	100.0	0.942717

4.5 Interpretation

Decoder choice is not a side issue. On the 10-bit pricing artifact at $\text{ell} = 3$, BP1 preserves most of the objective quality ($\text{best_F} = 2900.0$) while brute force collapses to 2300.0 , and it does so with an order-of-magnitude higher success probability. In structure-regime terms, BP1 is especially valuable in the middle regime where the artifacts are still decodable enough for the decoder to matter, but no longer trivial enough for all decoders to tie.

Verdict: Decoder sensitivity is one of the benchmark’s genuinely nontrivial results, and BP1 is the practical decoder to emphasize on the harder validated slices.

5. Resources, circuit realism, and Qiskit validation scope

Code: `resources.py`, `dicke_circuit.py`, `dqi_circuit_qiskit.py`, `06_resource_analysis.ipynb`, `07_qiskit_structural_parity.ipynb`, `13_scaling_analysis.ipynb`

5.1 Why this section matters

The industrial DQI paper places major emphasis on **circuitization**, **resource estimation**, and **small-instance circuit simulations**. This benchmark adopts that same expectation, but at smaller scale and with stricter scoping language. The goal is not to claim a full decoder-inclusive circuit implementation for industrial instances; it is to make the resource and parity boundary explicit.

5.2 Notebook 06 — analytic full-pipeline estimates vs Qiskit subset counts

Notebook 06 separates full-pipeline analytic estimates from transpiled structural-subset counts.

Quantity	Value
Same-model rows aligned on identity	140
Same-family evidence rows	79
Qiskit structural-subset payload available for completed rows	122 / 140 = 87.1%
Rows with subset payload absent	18

This distinction is important. It prevents the report from mixing full decode-inclusive analytic costs with subset-only transpilation results.

5.3 Notebook 07 — structural-subset parity

Notebook 07 is a successful **structural-subset parity** notebook.

Quantity	Value
Artifact	<code>S_sparse_low_collision.json</code>
Instance id	<code>S_sparse_low_collision</code>
m	12
n	10
ell	2
TVD	<code>0.000e+00</code>
Max absolute probability difference	<code>0.000e+00</code>
Syndrome qargs	<code>[4, 5, 6]</code>

The validated parity claim is explicitly restricted to:

- approximate low-weight superposition,
- phase kick,
- syndrome compute,
- final Hadamards,

and explicitly excludes decode/uncompute from the general parity claim.

5.4 Analytic resource estimates across P3–P7

Instance	n	m	ell	Total qubits	Syndrome CNOTs	Dicke-prep gates	Estimated depth	Validation scope
P3	6	15	2	48	33	68	2953	analytic + executed
P4	8	15	2	53	25	68	6333	analytic + executed
P5	10	15	2	57	27	68	8399	analytic + executed
P6	12	15	2	61	19	68	11097	analytic + executed
P7	14	15	2	65	19	68	12848	analytic-only

Timing and memory anchors for the executed pricing rows:

Instance	Encoding	Decoder	Alpha	ell	Mean wall-clock (s)	Peak memory (MB)
P3	wht_truncated_pricing	bpl	paper	1	0.629699	8.459208
P3	wht_truncated_pricing	bruteforce	paper	1	0.826408	8.463276
P4	wht_truncated_pricing	bpl	paper	2	2.885672	33.632772
P4	wht_truncated_pricing	bruteforce	paper	2	0.942717	33.654672
P5	wht_truncated_pricing	bpl	paper	3	22.272777	1074.309777
P5	wht_truncated_pricing	bruteforce	paper	3	14.114878	1074.452771

5.5 Interpretation

The resource story is now both stronger and more honest. Qiskit evidence exists and is positive, but it is carefully bounded to a structural subset. Full-pipeline costs are still presented, but as analytic estimates. The memory jump to roughly 1.07 GB by P5 makes the P7 analytic-only boundary credible rather than ad hoc.

Verdict: Resource accounting and Qiskit evidence are now reviewer-usable because the scope boundary is explicit.

6. Dynamic-pricing extension and scaling boundary

Code: problem_generator_multiperiod.py, pricing_model_ortools_multiperiod.py, 09_3period_pricing_extension.ipynb, 13_scaling_analysis.ipynb

6.1 Three-period frozen scenario matrix

Notebook 09_3period_pricing_extension.ipynb freezes a 9-row scenario matrix for a 3-period extension with shared inventory and time-varying demand weights.

Scenario	Dynamic revenue	Static revenue	Delta	Remaining inventory	Package changed across periods
low-tight-smooth-0	2100.0	2100.0	0.0	2	False
low-buffered-smooth-5	2100.0	2100.0	0.0	3	False
medium-tight-smooth-0	2104.0	2086.0	18.0	1	True
medium-tight-smooth-20	2086.0	2086.0	0.0	2	False
medium-buffered-smooth-0	2104.0	2086.0	18.0	2	True
medium-loose-smooth-5	2099.0	2086.0	13.0	2	True
high-tight-smooth-0	2236.0	2086.0	150.0	1	True
high-tight-smooth-5	2231.0	2086.0	145.0	1	True
high-buffered-smooth-20	2216.0	2086.0	130.0	2	True

Frozen instance summary:

Quantity	Value
Number of periods	3
Number of configurations	16
Initial inventory	3
Per-period capacity caps	[2, 2, 2]
Displayed smoothing penalty	5.0

6.2 Scaling boundary through P7

Notebook 13 extends the pricing family through P6 and P7 and makes the simulation boundary explicit.

Instance	Features	Bits	Optimal F*	Status
P6	6	12	3360.0	executed + analyzed
P7	7	14	3100.0	analytic-only

6.3 Interpretation

The three-period extension is useful because it demonstrates **behavioral nontriviality**, not just extra notation. In low-demand or heavily smoothed regimes the dynamic solution collapses to the static policy ($\Delta = 0$), while in medium and high-demand regimes it produces real horizon-level gains and changes the selected package across periods. The scaling notebook then closes the loop by showing

where the validated single-period statevector story currently stops.

Verdict: The project now has both a dynamic-pricing-adjacent extension and an explicit P7 execution boundary, which together make the benchmark feel complete rather than overclaimed.

7. Appendix layer: alpha validation, dashboarding, and quality vs cost

Code: 10_alpha_finite_size_validation.ipynb, 11_decision_faithfulness_dashboard.ipynb, 12_quality_vs_cost.ipynb

7.1 Alpha finite-size appendix (Notebook 10)

Notebook 10 is now a completeness-gated appendix rather than a speculative alpha comparison notebook.

Coverage gate:

Metric	Value
Total candidate rows examined	48
Rows retained after alpha-completeness filter	42
Rows excluded by completeness filter	6
Scope filter	fallback: family=C + matrix=matrix_b (alpha validation slice)

Retained-summary panel:

Instance	Alpha mode	Execution mode	Mean best F	Mean top-1 regret	Mean postselection success
C1	heuristic	coherent	4.0	0.0	1.000000
C1	paper	mixture	4.0	0.0	1.000000
C1	uniform	mixture	4.0	0.0	1.000000
C2	heuristic	coherent	600.0	0.0	1.000000
C2	paper	mixture	600.0	0.0	1.000000
C2	uniform	mixture	600.0	0.0	1.000000
C3	heuristic	mixture	6.0	0.0	0.839086
C3	paper	mixture	6.0	0.0	0.888393
C3	uniform	mixture	6.0	0.0	0.851190

Compact verdict table:

Instance	Verdict	Alpha modes with zero regret
C1		heuristic, paper, uniform
C2		heuristic, paper, uniform
C3		heuristic, paper, uniform

7.2 Decision-faithfulness dashboard (Notebook 11)

Shared metric panel for ilp_derived_exact:

Metric	Availability	Mean	Median
spearman_rho_F_G	6/6	1.0	1.0
retained_energy_eta	6/6	1.0	1.0
topk_regret	6/6	0.0	0.0
best_F	6/6	600.0	600.0
decision_distortion	6/6	0.0	0.0

Focused unavailable/skipped categories:

Category	Count
decoder_not_implemented	22
execution_failure	10
structural_boundary	4
expected_unavailable	3

Broader unavailability accounting:

Unavailability category	Count
structural_boundary	92
expected_missing_metric	75
resource_comparison_issue	58
other	38
dependency_issue	32

7.3 Quality vs cost appendix (Notebook 12)

Filter audit:

Step	Input rows	Output rows	Rows removed
pre-filter comparable slice	36	36	0
comparable encoding allowlist	36	36	0
mixture-first filter	36	27	9
coherent-off filter	27	27	0
decoder filter (bp1 / bruteforce)	27	27	0

Representative retained anchors:

Artifact family	Encoding	Best F	Gates	Depth	Status
C1	wht_exact	4	5136	615	completed
C2	ilp_derived_exact	600	7576	713	completed
P3	wht_truncated	3000	21174	1167	completed
P4	wht_truncated	2800	27473	1327	completed
P5	wht_truncated	3000	42073	1634	completed

7.4 Interpretation

These notebooks should remain appendices, but they are now useful appendices. Notebook 10 contains a real retained slice instead of an empty shell. Notebook 11 makes missingness and comparability boundaries explicit. Notebook 12 gives a small but persistent quality-vs-cost view over the admitted comparable slice.

Verdict: The appendix layer is now honest, quantitative, and worth keeping, but it should not replace the benchmark’s mainline evidence. ~

Summary

Deliverable	Test	Result	Status
Pricing baseline exact	CP-SAT vs brute force on P3–P5	All match; $F^* = 3000.0$ at P5	Validated
Paper-faithful DQI microscope	Five-step toy + pricing slice	Clean at 6-bit, partial at 8-bit, degrades at 10-bit	Honest scope
Encoding recommendation	Exact-reference vs WHT on 18 matched pairs	$\rho(F, G) = 1.0$ vs 0.9646 ; zero regret vs 88.89	Clear winner
Decoder sensitivity	BP1 vs brute force at $\ell_1 = 3$	BP1: $F = 2900.0$, success 0.769597 ; BF: $F = 2300.0$, success 0.075275	Nontrivial
Qiskit structural parity	Structural subset TVD	$TVD = 0.0$	Validated
Resource scaling P3–P7	Analytic qubit/depth estimates	$48 \rightarrow 65$ qubits, depth $2953 \rightarrow 12848$	Scoped
Dynamic extension	3-period scenario matrix	Uplift 0 to 150 across 9 scenarios	Real behavior
Alpha appendix	Completeness-gated Family C slice	C1/C2/C3 all zero-regret across retained alpha modes	~ Appendix quality
Scaling boundary	P-family through P7	P6 executed, P7 analytic-only	Bounded

8. Bridge: connecting the benchmark to the DQI literature

This benchmark is easiest to understand when positioned explicitly between the two cited DQI papers.

Aspect	Jordan et al. (arXiv:2408.08292)	Sabater et al. (arXiv:2509.08328)	This benchmark
Core algorithmic object	DQI for max-XORSAT / max-LINSAT with polynomial filtering and decoding	Industrial vehicle pricing ILP reduced to max-XORSAT and benchmarked with DQI framing	Frozen pricing benchmark with explicit artifacts and exact-reference validation
DQI structure emphasized	Five-step state preparation, decode/uncompute, final Hadamard and measurement	Same DQI backbone, with strong emphasis on decoder circuitization and industrial benchmark design	Five-step microscope retained explicitly in notebooks and source modules
Weight choice	Paper-derived eigenvector weights w from $A(m, ell, d)$ when applicable	Uses DQI framing but emphasizes practical solver pipeline	paper, heuristic, and uniform alpha modes compared explicitly
Decoder story	Imperfect decoding and postselection are part of the theory	Belief-propagation implementation is central to industrial positioning	BP1 vs brute-force sensitivity is measured quantitatively on pricing and structure slices
Resource story	Algorithmic / theoretical scaling and decoder assumptions	Detailed circuit and logical resource emphasis for industrial down-scaled instances	Small-scale analytic resource accounting + structural-subset Qiskit parity
Main scientific role	Foundational DQI theory and algorithm design	Industrial problem framing and end-to-end ILP relevance	Exact-verification “microscope” and reviewer-facing benchmark packaging

8.1 Reviewer-facing interpretation of that bridge

The strongest claim supported by this repository is not “industrial DQI solved end to end.” It is more precise and more credible:

A compact, quantitatively validated pricing-to-DQI benchmark suite that faithfully reproduces the DQI structure at small scale, demonstrates an exact-reference reduction path for pricing artifacts, quantifies decoder/postselection sensitivity, and states its circuit/resource scope explicitly.

That is the right way to connect the benchmark to the cited literature:

- **Jordan et al.** supplies the algorithmic backbone and the language for weights, decoder failure, and postselection.
- **Sabater et al.** supplies the industrial pricing framing, the ILP-to-max-XORSAT narrative, and the expectation that resource/circuit realism must be addressed.
- **This benchmark** supplies the exact-checkable, notebook-backed validation layer between those two.

9. Reviewer-facing assessment

9.1 Strongest points

Strength	Evidence
Pricing-to-DQI bridge is concrete	Notebook 04 solves the pricing instance exactly and exports both mainline and exploratory artifacts
Exact-reference encoding clearly wins	Notebook 08 shows lower regret and higher faithfulness on 18 matched pricing records
Decoder story is nontrivial	Notebooks 05, 09, and 13 show BP1 gains in harder regimes
Resource story is properly scoped	Notebook 06 separates analytic full-pipeline estimates from Qiskit subset counts
Qiskit evidence is honest and positive	Notebook 07 shows successful structural-subset parity with $TVD = 0$
Scaling story is explicit	Notebook 13 extends the family to P6/P7 and states the P7 analytic-only boundary
Dynamic extension is real	Notebook 09_3period freezes multiple nonzero-uplift scenarios

9.2 What should remain secondary or carefully phrased

Topic	Recommended phrasing
Notebook 03	“Exploratory DQI microscope” rather than “mainline pricing benchmark”
Notebook 07	“Structural-subset Qiskit parity” rather than full end-to-end DQI parity
Notebook 10	“Compact appendix over complete retained rows” rather than a headline alpha study
Notebook 11	“Transparency dashboard” rather than benchmark centerpiece
Notebook 12	“Reviewer-facing appendix over admitted comparable rows” rather than broad performance summary

9.3 Suggested presentation order

- 04_pricing_pipeline.ipynb
- 08_paired_encoding_comparison.ipynb
- 05_decoder_comparison.ipynb
- 13_scaling_analysis.ipynb
- 07_qiskit_structural_parity.ipynb
- 09_3period_pricing_extension.ipynb
- 06_resource_analysis.ipynb

8. 09_decoder_phase_diagram.ipynb
 9. 01, 02, 03, 10, 11, 12 as supporting or appendix material
-

References

1. L. Jordan et al., *Optimization by Decoded Quantum Interferometry*, arXiv:2408.08292.
 2. F. Sabater et al., *Towards solving industrial integer linear programs with Decoded Quantum Interferometry*, arXiv:2509.08328.
 3. Internal formatting reference: Continuum_Physics_Results_and_Validation copy.md.
-

Bottom-line statement

Taken together, the 14 uploaded notebooks support a credible and literature-aligned benchmark story:

- the pricing benchmark is well-defined and exactly solved at baseline;
- the pricing-to-artifact handoff is explicit;
- the DQI implementation is framed in the language of the original five-step algorithm;
- exact-reference / ILP-derived encoding is the correct mainline backend;
- decoder choice materially affects postselection and achieved objective quality;
- Qiskit evidence is positive when phrased correctly as **structural-subset parity**;
- resource and scaling boundaries are explicit through P7; and
- the 3-period extension demonstrates real dynamic-pricing-adjacent behavior rather than a cosmetic variation.

The strongest overall claim is therefore:

A compact, quantitatively validated pricing-to-DQI benchmark suite with exact baselines, a clear mainline encoding recommendation, decoder sensitivity analysis, structural-subset Qiskit validation, and an explicit scaling/resource boundary through P7, written in the paper-faithful language of DQI and positioned as a verified bridge between foundational DQI theory and industrial pricing formulations.